

TP3 : Introduction à l'algorithmique et à Python

NOM ETUDIANT :

Types structurés, Sous-programmes et Programme principal

Chaque notebook de TP est à déposer sur moodle dans la zone de dépôt dédiée, au plus tard, la semaine suivant le TP ou selon la date spécifiée sur moodle. Ce dépôt tient lieu de compte-rendu et sera évalué.

Le nom de votre fichier .ipynb doit contenir votre nom. Exemple : TP1_PYTHON_DUPONT.ipynb

Bonnes pratiques :

- les noms de variables et de constantes doivent être explicites sauf s'il existe des conventions par exemple i,j,k pour les indices ;
- le code doit être commenté lorsque les actions réalisées sont complexes les commentaires doivent refléter l'algorithme sous-jacent ;
- chaque programme doit faire l'objet des tests des cas limite et d'un rapport de tests tester le fonctionnement normal, ainsi que les cas d'erreur potentiels.

Le code est écrit dans une cellule code et les rapports de tests par copie-coller des résultats des tests successifs, et ce, en dessous du texte de l'exercice.

Un rapport de test s'accompagne de message explicitant la demande de saisie ou bien la nature de la valeur affichée. On doit comprendre la nature du test effectué.

Entrée []:

```
# A NOTER
# les commandes UNIX à exécuter depuis un notebook doivent être précédées de !
# pour que le changement de répertoire soit effectif la commande cd doit être précédée de !
# vérifier où on se trouve dans l'arborescence des fichiers ... quelques commandes utiles
!pwd; ls;
#!mkdir TP1_VOTRE_NOM
#!cd chemin_vers_le_répertoire
#!pwd; ls;

# marque un commentaire
```

PARTIE 1 - Définition et gestion des éléments constituant l'environnement du robot

EXERCICE 1 : possibilités de la bibliothèque turtle

Importer la bibliothèque avec la commande

```
import turtle as tl
```

Contexte : un robot évolue dans une pièce, comportant d'éventuels obstacles.

- Une pièce est représentée par ses dimensions (longueur, largeur), la position du coin haut droit (x,y), la position des ouvertures (x,y,largeur_porte).
- Un obstacle est représenté par la position de son centre, son type (cercle ou carré) et la dimension correspondante (rayon ou côté)

A l'aide de la bibliothèque turtle, on veut représenter la pièce, ses ouvertures et ses éventuels obstacles avant de faire évoluer le robot (flèche turtle) dans cet espace en évitant les obstacles présents.

Exécuter successivement les blocs de code suivants (dans l'ordre). Analyser le code et son résultat.

Entrée []:

```
#1#  
# bibliothèque nécessaires au TP  
import turtle as tl  
import random as rd
```

Entrée []:

```
#2#
# Prise en compte d'une pièce

# nettoyage de la table traçante
tl.clearscreen()

# Stylet en position haute - pas de traçage
tl.up()

# position du coin Haut Droit
coin_HD = (200, 200)
x_HD, y_HD = coin_HD
print('TRACE coordonnées coinHD =', x_HD, y_HD)

# déplacement au coin Haut Droit
tl.goto(coin_HD)

# direction à prendre pou le traçage du premier côté
tl.right(90)

# Dimensions de la pièce carrée
longueur_cote = 400
angle = 90

# Données pour le traçage
tl.width(5)                # largeur du trait
tl.color('blue')           # couleur du trait
tl.down()                  # stylet en position basse pour le traçage

# tracer les contours de la pièce
for i in range(4):
    tl.forward(longueur_cote)
    tl.right(angle)

tl.up()                    # retour en position haute
```

Entrée []:

```
#3#
# Prise en compte d'une ouverture

# caractéristiques de la porte 1
largeur_portel = 60
distance_portel_mur_droit = 100
pos_portel = (x_HD, y_HD - distance_portel_mur_droit)
couleur_portel = 'orange'
largeur_trait_portel = 10
print('position portel = ', pos_portel)

# Traçage de la porte 1
tl.width(10)                # largeur du trait des ouvertures
tl.up()                     # stylet en position haute
tl.forward(distance_portel_mur_droit) # déplacement jusqu'à la position de la porte

tl.down()                   # stylet en position basse
tl.width(largeur_trait_portel)
tl.color(couleur_portel)
tl.forward(largeur_portel)  # traçage de la porte
tl.up()                     # retour en position haute
```

Entrée []:

```

#4#
# Prise en compte d'un obstacle

# caractéristiques de l'obstacle
type_obstacle1 = 'cercle'
rayon_obstacle1 = 20
delta_pos_x = 150
delta_pos_y = 200
pos_obstacle1 = (x_HD - delta_pos_x , y_HD - delta_pos_y)

couleur_obstacle1 = 'green'
largeur_trait_obstacle1 = 10

print('position obstacle 1 = ', pos_obstacle1)

# Traçage de l'obstacle
tl.width(largeur_trait_obstacle1)                # largeur du trait des ouvertures
tl.up()                                           # stylet en position haute
tl.goto(pos_obstacle1)                          # déplacement jusqu'au la position

tl.width(largeur_trait_obstacle1)
tl.color(couleur_obstacle1)
if type_obstacle1 == 'cercle' :
    tl.forward(rayon_obstacle1)
    tl.down()                                     # stylet en position basse
    tl.begin_fill()
    tl.circle(rayon_obstacle1)
    tl.end_fill ()

tl.up()                                           # retour en position haute

```

Entrée []:

```

#5#
# positionnement du robot à l'extérieur de la pièce
delta_robot_x = -50
delta_robot_y = distance_portel_mur_droit + 30
pos_robot = (x_HD - delta_robot_x, y_HD - delta_robot_y)
print('TRACE position robot = ', pos_robot)
tl.goto(pos_robot)

```

Entrée []:

```
#6#
#déplacement du robot dans la pièce

# caractéristique du robot
couleur_robot = 'red'
largeur_trait_robot = 3
distance_parcours1 = 100

# orientation du robot
tl.right(angle)
tl.width(largeur_trait_robot)
tl.color(couleur_robot)
tl.down()
tl.forward(distance_parcours1)
tl.up()

#
```

EXERCICE 2 : Types structurés natifs : list, tuple, set, dict et sous-programmes

On veut structurer les données de façon à représenter chaque élément pièce, ouverture, obstacle, robot en regroupant leurs caractéristiques respectives.

Le point de coordonnées (0,0) est le centre de la table traçante.

Exercice 2.1 : représentation d'une pièce

Une pièce se caractérise par :

- un nom
- ses dimensions (longueur, largeur)
- la position du coin en haut à droite (x_HD, y_HD)
- la liste des ouvertures (renseigné plus tard)
- la liste des obstacles (renseigné plus tard)

PARTIE A.1

Définir la structure de données la plus appropriée selon vous pour représenter une pièce. Justifiez votre choix dans la zone de commentaire. Dans une nouvelle cellule, réaliser la séquence de code qui permet de faire les choses suivantes :

- a) Définir alors la variable `pièce1`,
- b) Afficher son type, son identifiant et sa valeur pour vérification (mode TRACE) ainsi que celles de ses constituants
- c) Vérifier la cohérence des informations dimensions et coordonnées coind HD/longueur/largeur
- d) Afficher si la pièce est carrée ou pas.
- e) Créer la variable `coin_piece` qui contient les coordonnées des 4 coins de la pièce1 (HD, BD, BG, HG). Les valeurs sont générées automatiquement à partir des la donnée 'coin_HD' de la pièce considérée. prendre en compte que faire le tour de la pièce c'est passer successivement d'un coin au suivant dans le sens des aiguilles d'une montre. Justifier le choix de la représentation

Vous inclurez des messages de TRACE pour voir les différentes étapes et construirez votre rapport de test en fonction des exécutions réalisées.

Différencier les affichages :

- mode TRACE : `print('TRACE message approprié', valeur)` pour vérifier la cohérence de la valeur affichée par rapport à ce qu'il est censé se passer. Destiné au développeur.
- mode normal `print('Message utilisateur', valeur )` pour informer l'utilisateur

COMMENTAIRES et RAPPORT DE TESTS ICI.

On peut définir un dictionnaire pour nommer chaque caractéristique

- un nom --> chaîne de caractères
- ses dimensions --> tuple (longueur, largeur)
- la position du coin en haut à droite --> tuple (x_{HD}, y_{HD})
- la liste des ouvertures --> liste vide au début
- la liste des obstacles --> liste vide au début

RAPPORT DE TESTS

Entrée []:

#A.1#

PARTIE A.2

Reprendre dans une cellule séparée, le code de la cellule traçant la pièce et l'adapter en fonction de la structure que vous aurez proposée dans la partie 1. Faire une capture écran du résultats que vous enregistrerez dans votre dossier TP3_NOM_ETUD pour quelle fasse partie du dépôt de l'archive du notebook.

COMMENTAIRES et RAPPORT DE TESTS ICI.

PARTIE A.3

Ecrire le sous-programme `tracer_piece` qui permet de tracer les contours de la pièce passée en paramètre. a) Quel est le type de ce sous-programme ? Fonction ou procédure ? b) tester ce sous-programme dans une cellule à part

POUR CHAQUE SOUS-PROGRAMME ECRIT, respectez systématiquement les consignes de vérification suivantes :

- Consigne 1 : Avant l'appel du sous-programme, affichez dans un message de trace, l'identifiant de chaque paramètre formel
- Consigne 2 : En début de sous-programme affichez dans un message de trace, l'identifiant de chaque paramètre formel. Vérifier la cohérence.
- Consigne 3 : après la définition de la fonction écrire du code de test pour vérifier son bon fonctionnement et garder la trace des tests effectués (en commentaire)

COMMENTAIRES et RAPPORT DE TESTS ICI.

Exercice 2.2 : représentation d'une ouverture

Une ouverture se caractérise par : - un nom - sa largeur - le coin gauche du mur sur lequel l'ouverture se trouve (renseigné plus tard) - la distance au coin du mur - la position de la porte (renseigné plus tard) - la couleur avec laquelle elle sera tracée - l'épaisseur du trait avec laquelle elle sera tracée

PARTIE B.1

Définir la structure de données la plus appropriée selon vous pour représenter une ouverture. Justifiez votre choix dans la zone de commentaire.

- a) Définir alors la variable `ouverture1`,
- b) Afficher son type, son identifiant et sa valeur pour vérification (mode TRACE) ainsi que celles de ses constituants
- c) La position de l'ouverture sera renseignée une fois celle-ci tracée (voir partie B.2)

COMMENTAIRES et RAPPORT DE TESTS ICI.

dictionnaire

```
TRACE piece1 : <class 'dict'> 140500746022784 {'nom': 'porte1', 'largeur': 60, 'distance_porte_angle': 100, 'pos_porte': (), 'couleur': 'orange', 'trait': 10} TRACE type et id = nom <class 'str'> 140500716734704 TRACE type et id = largeur <class 'int'> 140500643676368 TRACE type et id = distance_porte_angle <class 'int'> 140500643677648 TRACE type et id = pos_porte <class 'tuple'> 140500643475520 TRACE type et id = couleur <class 'str'> 140500718908336 TRACE type et id = trait <class 'int'> 140500643486288
```

Partie B.2

Reprendre ci-après le code de la cellule traçant l'ouverture et l'adapter en fonction de la structure que vous aurez proposée. Vous inclurez des messages de TRACE pour voir les différentes étapes et construirez votre rapport de test en fonction des exécutions réalisées. La position de la porte (position atteinte en fin de tracé) sera mise à jour après le tracé.

Faire une capture écran du résultats que vous enregistrerez dans votre dossier TP3_NOM_ETUD pour quelle fasse partie du dépôt du notebook.

COMMENTAIRES et RAPPORT DE TESTS ICI.

PARTIE B.3

Ecrire le sous-programme `ajouter_ouverture` qui permet d'ajouter l'ouverture passée en paramètre à la liste des ouvertures de la pièce passée également en paramètre.

- a) Quel est le type de ce sous-programme ? Fonction ou procédure ?
- b) tester ce sous-programme dans une cellule à part

CONSIGNES A APPLIQUER cf section PARTIE A3.

Ecrire le sous-programme `supprimer_ouvertures` qui réinitialise la liste des ouvertures de la pièce passée également en paramètre, à la liste vide

- a) Quel est le type de ce sous-programme ? Fonction ou procédure ?
- b) tester ce sous-programme dans une cellule à part.

COMMENTAIRES et RAPPORT DE TESTS ICI.

Partie B.4

Ecrire le sous-programme `tracer_ouvertures_piece` qui permet d'afficher les ouvertures de la piece passée en paramètre.

- a) Quel est le type de ce sous-programme ? Fonction ou procédure ?
- b) tester ce sous-programme dans une cellule à part, afficher la valeur avant et après l'appel.

APPLIQUER LES CONSIGNES DE VERIFICATION ...

COMMENTAIRES et RAPPORT DE TESTS ICI.

Exercice 2.3 : représentation d'un obstacle

Un obstacle se caractérise par :

- un nom
- son type (carré ou rond)
- la position de son centre relativement au centre de l'espace (0,0)
- la dimension du rayon ou du côté
- la couleur
- l'épaisseur du trait

PARTIE C.1

Définir la structure de données la plus appropriée selon vous pour représenter un obstacle (carré ou rond). Justifiez votre choix dans la zone de commentaire.

- a) Définir alors la variable `obstacle1`,
- b) Afficher son type, son identifiant et sa valeur pour vérification (mode TRACE) ainsi que celles de ses constituants
- c) Avant de le tracer, vérifier si l'obstacle est bien dans la pièce, et qu'il ne touche pas le mur (distance de +/-5 respectée). indiquer que l'obstacle ne peut être placé si non.

COMMENTAIRES et RAPPORT DE TESTS ICI.

PARTIE C.2

Reprendre dans une cellule ci-dessous le code de la cellule traçant un obstacle et l'adapter en fonction de la structure que vous aurez proposée.

COMMENTAIRES et RAPPORT DE TESTS ICI.

PARTIE C.3

Ecrire le sous-programme `ajouter_obstacle` qui permet d'ajouter un obstacle passé en paramètre à la liste des obstacles de la pièce passée également en paramètre.

- a) Quel est le type de ce sous-programme ? Fonction ou procédure ?
- b) tester ce sous-programme dans une cellule à part

CONSIGNES A APPLIQUER cf section PARTIE A3.

Ecrire le sous-programme `supprimer_obstacle` qui réinitialise la liste des obstacles de la pièce passée également en paramètre, à la liste vide

- a) Quel est le type de ce sous-programme ? Fonction ou procédure ?
- b) tester ce sous-programme dans une cellule à part.

COMMENTAIRES et RAPPORT DE TESTS ICI.

PARTIE C.4

Ecrire le sous-programme `tracer_obstacles_piece` qui permet d'afficher les obstacles de la pièce passée en paramètre :

- a) Quel est le type de ce sous-programme ? Fonction ou procédure ?
- b) tester ce sous-programme dans une cellule à part, afficher la valeur avant et après l'appel.

APPLIQUER LES CONSIGNES DE VERIFICATION ET DE TEST ...

COMMENTAIRES et RAPPORT DE TESTS ICI.

PARTIE 2 - Définition et gestion du robot et de ses déplacements

EXERCICE 3 : représentation du robot

Un robot est représenté par la flèche sur la table traçante. Il se caractérise par

- une nom
- une couleur
- la largeur du trait qui représente ses déplacements
- une orientation initiale (Est = 0, Nord = 1, Ouest = 2, Sud = 3). Multiplié par 90 cela donnera l'angle
voir les fonctions `tl.heading` et `tl.setheading` de `turtle`
- la position (`x_robot`, `y_robot`) une fois placé dans l'environnement
- la liste des tronçons de trajectoire à réaliser (non renseignée initialement)

Partie D.1

Définir la structure de données la plus appropriée selon vous pour représenter un robot. Justifiez votre choix dans la zone de commentaire.

- a) Définir alors la variable robot,
- b) Afficher son type, son identifiant et sa valeur pour vérification (mode TRACE) ainsi que celles de ses

Partie D.2

Reprendre ci-après le code de la cellule traçant le déplacement du robot et l'adapter en fonction de la structure que vous aurez proposée. Vous incluez des message de TRACE pour voir les différentes étapes et construirez votre rapport de test en fonction des exécutions réalisées.

- Vérifier que le robot est bien en dehors de la pièce au démarrage avant de procéder au déplacement.
- Il doit être à 5 pas de tout mur extérieur.

On vérifiera plus tard (suite TP) :

- qu'il est bien face à une ouverture pour pouvoir entrer.
- et qu'il n'y a pas d'obstacles sur son chemin

COMMENTAIRES et RAPPORT DE TESTS ICI.

PARTIE D.3

Une trajectoire est une liste de segments. Un segment est caractérisé par :

- a) son point de départ
- b) son orientation (0,1, 2, ou 3), sens dans lequel le robot doit avancer (cf. définition du robot)
- c) la distance à parcourir
- d) son point d'arrivée (renseigné une fois le déplacement fait) avec `tl.pos()`

Exemple de Trajectoire 1 :

- Segment 1 : départ (250,90) ; direction = 2 ; distance = 100 ; arrivée = ()
- Segment 2 : départ (150, 90) ; direction = 1 ; distance = 50 ; arrivée = ()
- Segment 3 : départ (150, 140) ; direction = 2 ; distance = 250; arrivée = ()
- Segment 4 : départ (-100, 140) ; direction = 1 ; distance = 100; arrivée = ()
- Segment 5 : départ ((-100, 40) ; direction = 0 ; distance = 100 ; arrivée = ()

1) Ecrire le sous-programme `ajouter_trajectoire` qui permet d'ajouter une trajectoire au robot passé en paramètre.

2) Ecrire le sous-programme `supprimer_trajectoire` qui réinitialise la trajectoire du robot à la liste vide.

CONSIGNES A APPLIQUER cf section PARTIE A3.

COMMENTAIRES et RAPPORT DE TESTS ICI.

Entrée []:

```
# test à faire

print('TRACE identifiant robot = ', id(robot))
trajectoire = [ [(250,90), 2, 100, ()], [(150,90), 1, 50, ()], [(150, 140), 2, 250,
                    [(-100, 140), 3, 100, ()], [ (-100, 40), 0, 100, (]] ]

ajouter_trajectoire(robot, trajectoire)

print('TRACE valeur robot = ', robot)
print('-----')
```

PARTIE D.5

Un robot se déplace dans un environnement composé d'une pièce, de ses ouvertures et de ses obstacles. Le déplacement se fait selon une trajectoire constituée d'une succession de segments.

On positionnera systématiquement le robot en face de la première ouverture de la pièce et orienté dans le sens lui permettant de rentrer dans la pièce.

1) Ecrire le sous-programme `modifier_orientation` qui :

- détermine l'angle d'orientation courant du pointeur représentant le robot
- vérifie si une modification d'orientation est nécessaire par rapport à l'orientation passée en paramètre
- et suivant le cas modifie ou pas l'orientation courante

=> utiliser les fonctions `tl.heading()` qui donne l'orientation courante et `tl.setheading(angle)` qui la modifie selon l'angle indiqué.

2) Ecrire le sous-programme `positionner_robot` qui positionne le pointeur représentant le robot à la position et dans le sens qui correspondent au robot passé en paramètre (`pos_robot` et `sens`).

3) Ecrire le sous programme `executer_trajectoire` qui déplace le robot passé en paramètre suivant la trajectoire qui lui a été associée. Cette trajectoire est à réaliser dans l'environnement correspondant à la pièce donnée en paramètre.

4) Suivant le cas, il sera peut-être nécessaire d'écrire un sous-programme `valiser_position` pour vérifier que la position courante et la position de début de segment sont suffisamment proches à +/- delta (delta passé en paramètre).

Entrée []:

```
# test à faire
executer_trajectoire(robot, piece1)
```

Entrée []:

